

Flink 第1讲

已经有 Spark 了，为什么还需要 Flink?

从批处理走向真正的流处理：无界数据、Event Time、Watermark、Window、State 与 Checkpoint

有界数据与无界数据：Flink 从这里开始



Batch 本质上是在“等数据集完整处理后”；Streaming 则必须在数据仍不断到来时持续计算。

Flink 的核心思想：把 bounded 和 unbounded 都视为“流”，批处理只是有界流的一种特殊情况。

本讲目标：先建立 Flink 的“问题意识”，暂时不被 API 和代码细节干扰。

课程默认讲稳定、通用的 Flink 流处理概念，不围绕实验性 API 展开。

一、先回答最根本的问题：为什么还需要 Flink？

学完 Spark 后，很容易产生一个合理疑问：Spark 已经有 Structured Streaming，也能实时处理数据，那为什么还需要专门学 Flink？答案不是“Flink 更新”或“Flink 一定更快”，而是：Flink 的设计中心从一开始就是连续的、无界的、有状态的流计算。

Spark 已经能做流处理，为什么还要 Flink？

Spark：通用大数据计算平台
批处理/SQL生态非常强

- RDD / DataFrame / SQL
- 批分析、ETL、ML生态
- Structured Streaming 也能处理流

Flink：以流为第一等公民
围绕连续、有状态、事件时间计算设计

- Event Time / Watermark
- State / Timer / Checkpoint
- 低延迟、持续、复杂流逻辑

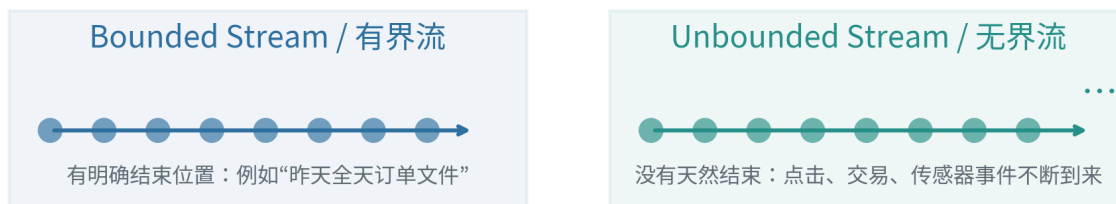
关键不是“谁绝对更强”，而是设计中心不同：Flink 从一开始就把“无界数据 + 状态 + 事件时间 + 故障恢复”当作核心问题。

Spark 非常强，尤其在批处理、SQL、ETL、数据湖和通用数据分析领域。Flink 的优势场景通常是：事件持续不断到来，需要低延迟处理；业务逻辑依赖历史状态；必须正确处理乱序事件；还希望故障恢复后业务语义尽量保持一致。

所以不要把两者理解成“老技术 vs 新技术”。更准确的理解是：设计中心不同，最佳适用场景不同。

二、Flink 的第一块地基：Bounded 与 Unbounded

有界数据与无界数据：Flink 从这里开始



Batch 本质上是在“等数据集完整后再处理”；Streaming 则必须在数据仍不断到来时持续计算。

Flink 的核心思想：把 bounded 和 unbounded 都视为“流”，批处理只是有界流的一种特殊情况。

2.1 有界数据 (Bounded)

有明确的开始和结束。例如“2026年9月28日全天订单.csv” “过去三年日志文件” “一张已经固定下来的 Hive 表”。你可以等数据全部存在后再计算，因此天然适合 Batch Processing。

2.2 无界数据 (Unbounded)

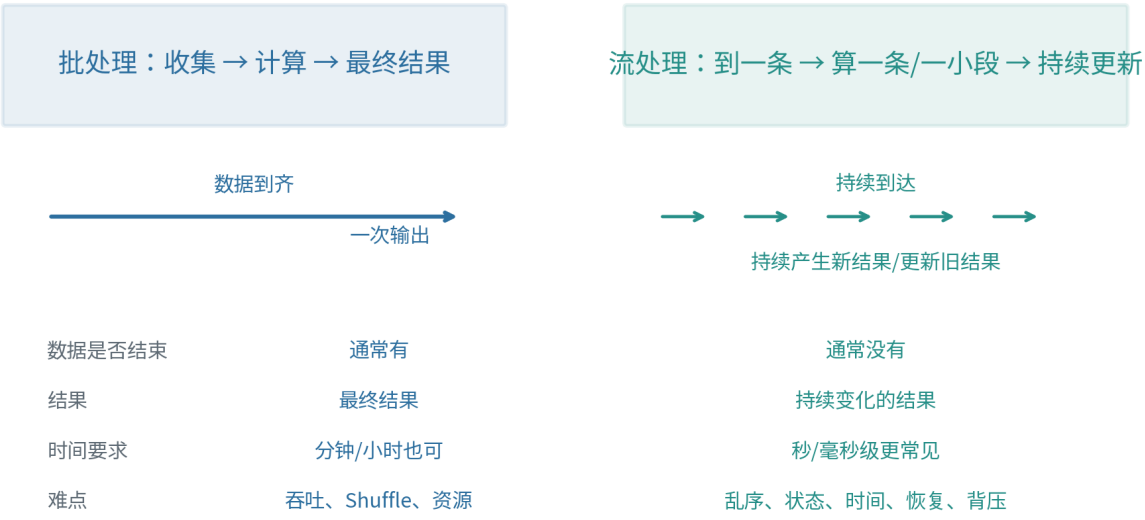
没有天然的结束，例如实时点击、银行卡交易、股票成交、服务器日志、物联网传感器数据。你永远不能说“所有数据终于到齐了”，因为下一秒还会继续来。

2.3 Flink 的统一视角

Flink 的重要思想是：所有数据都可以看成流；批处理只是处理一个最终会结束的有界流。这样很多运行时机制可以统一，而不必把“批”和“流”当成两个完全不同的系统。

三、批处理与流处理：区别绝不是“快一点”

批处理和流处理：区别不只是“快一点”



问题	批处理	流处理
输入	通常是已完成的数据集	数据持续到来，没有终点
结果	常常追求最终结果	结果持续产生、持续更新
延迟	分钟/小时可能可接受	经常要求秒级甚至更低
真正难点	吞吐、Shuffle、资源利用	乱序、时间语义、状态、容错、背压

举个最直观的例子：

批处理：
昨天一整天的订单已经落盘
→ 今天凌晨统一统计每个商品销售额

流处理：
订单一笔一笔不断产生
→ 仪表盘希望每 5 秒更新一次销售额
→ 风控希望某笔可疑交易到来后 200ms 内报警

因此“流处理”并不是把原来的离线程序运行得更频繁，而是问题本身发生了变化。

四、现实场景：哪些问题天然像 Flink？

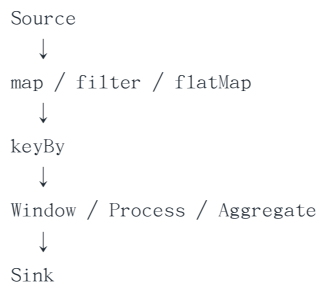
场景	连续事件	系统需要持续记住什么
支付风控	每一笔交易	用户过去几分钟金额、地点、失败次数
电商大屏	点击/下单/支付	当前窗口内 PV、UV、GMV
IoT 监控	温度、压力、振动	设备最近状态、连续异常持续时间
广告计费	曝光、点击、转化	用户/广告维度计数与去重状态
网络安全	登录、访问、告警	IP/账号行为序列、规则状态
金融行情	报价与成交	窗口指标、订单簿/策略状态

这些问题有一个共同特点：当前事件是否异常，往往取决于之前发生过什么。这就是为什么 Flink 课程很快就会碰到 State。

例如“同一账号 1 分钟内在 3 个国家登录”无法只看当前这一条记录判断；系统必须记住该账号最近一段时间的登录状态。

五、Flink 的基本执行直觉：数据像流水一样穿过 Operator

可以先把 Flink 程序想成一条持续运行的数据生产线：



Source 不断产生事件；Operator 逐条或逐小批处理；需要同 Key 数据聚合时通过 keyBy 重新分区；有状态算子保留历史状态；最终结果持续写到 Kafka、数据库、文件系统或其他 Sink。

和传统批处理最大的心理差别是：程序不是“读完一个文件就结束”，而是可以长期运行几天、几个月甚至几年。

这也带来三个新问题：

问题	为什么批处理中没那么突出
什么时候算一个时间段结束？	批数据已经完整；流数据永远在继续
机器重启后历史状态怎么办？	批任务可以重跑；流任务可能不能从头重算几个月
下游处理不过来怎么办？	流持续涌入，会形成 Backpressure

六、流处理最难懂的第一件事：到底按哪个“时间”算？

Event Time 与 Processing Time：到底按哪个“时间”算？



网络延迟、重试、移动网络抖动会导致：后发生的事件可能先到，先发生的事件反而后到。

6.1 Processing Time（处理时间）

指事件真正到达 Flink、被机器处理时的系统时钟时间。它简单、延迟低，但会受到网络抖动、消息积压、重试、机器负载影响。

6.2 Event Time（事件时间）

指事件在现实世界中真正发生的时间，通常已经写在数据字段中，例如订单创建时间、传感器采样时间、用户点击时间。

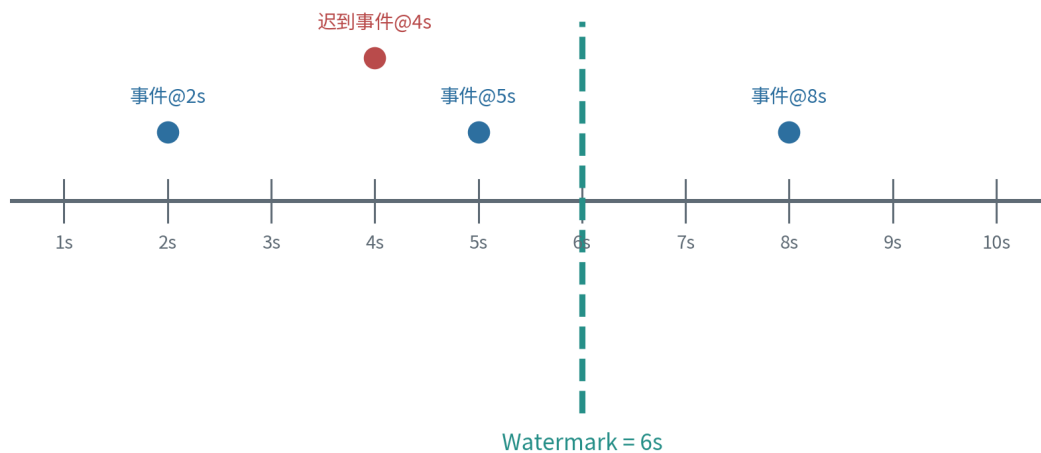
6.3 为什么 Event Time 很重要？

假设订单 B 在 10:00:04 发生，但由于移动网络延迟直到 10:00:12 才到 Flink。如果你问“10:00:00~10:00:10 实际发生了多少订单”，就应该按照 Event Time 放入窗口，而不是按照它什么时候到达服务器。

这一步是理解 Flink 的分水岭：系统时间不一定等于业务世界的时间。

七、乱序之后怎么办？Watermark 出场

Watermark：系统如何判断“这个时间段大概收齐了”？



直觉：Watermark 不是“绝对保证”，而是关于 Event Time 进度的声明/估计；越保守越等得久，越激进越容易出现 Late Event。

如果按照 Event Time 统计，马上会遇到一个棘手问题：你怎么知道“某个时间段的数据已经收齐了”？流永远不会结束，而且旧事件可能迟到。

Watermark 可以先用一句通俗话理解：

“根据我目前看到的数据，我认为事件时间已经推进到这里了；比这个时间更早的数据，理论上大部分应该已经到了。”

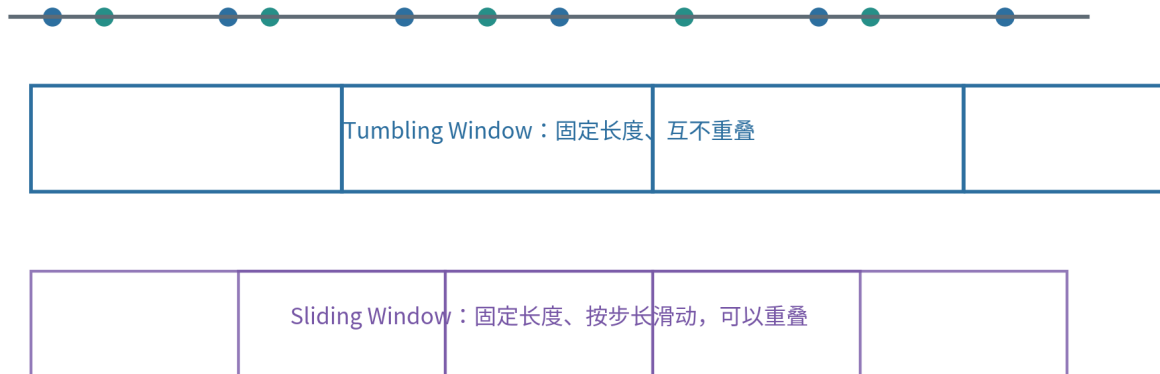
注意：Watermark 通常不是数学上的绝对保证，而是一种关于 Event Time 进度的机制。现实数据可能仍有 Late Event。设计 Watermark 就是在做一个权衡：

策略	优点	代价
等得更久/更保守	迟到事件更少，结果更完整	结果出来更晚
推进更快/更激进	低延迟	迟到事件更多，需要额外处理

这就是流处理里非常典型的权衡：Latency（延迟） vs Completeness（完整性）。

八、Window：无限的流，怎么计算“这一段”？

Window：无限数据流怎样变成“可以计算的一段”？



Session Window：按“活动间隔”动态切分，适合用户会话、设备活跃区间等。

如果数据没有结束，就无法像批处理那样直接“等全体数据后 SUM”。因此经常需要把无限流切成有限范围。Window 就是这个“切片工具”。

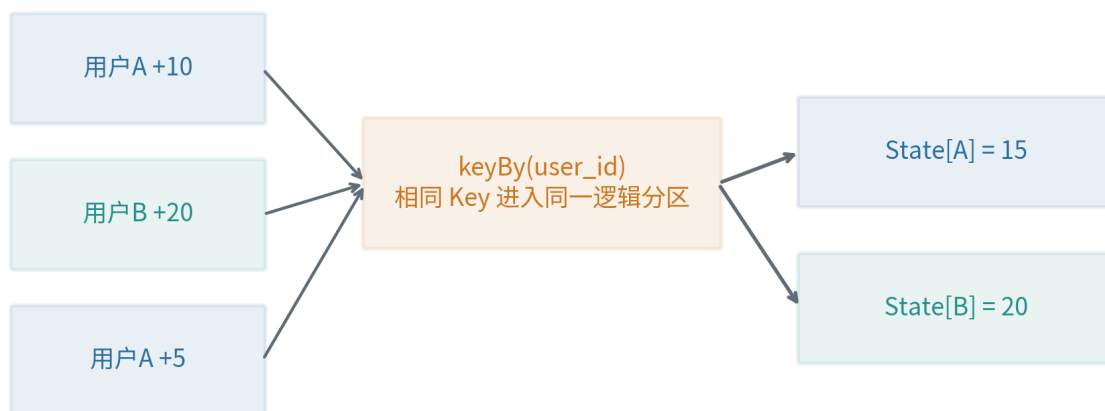
窗口	直觉	典型例子
Tumbling Window	固定长度且互不重叠	每 1 分钟统计一次销售额
Sliding Window	固定长度但按更小步长滑动，彼此重叠	统计最近 10 分钟、每 1 分钟更新一次
Session Window	按活动间隔动态划分	一次用户会话、一次设备活跃周期

例如“最近 5 分钟每个 IP 的请求次数”其实已经隐含三个条件：Key 是 IP；时间范围是 5 分钟；每个窗口里需要一个计数状态。

所以 Window 并不是一个孤立 API，它会和 Event Time、Watermark、State 一起工作。

九、State: Flink 为什么是“有记忆”的计算系统

State：流处理为什么必须“记住过去”？



没有 State，就只能“看见当前这一条”；有了 State，系统才能做累计计数、会话、风控规则、去重、模式识别等。

批处理中我们经常可以重新读取整张表再统计；长期运行的流系统不能每条事件都从几个月数据重算，因此必须把“过去计算到哪里”保存在状态里。

典型 State 场景：

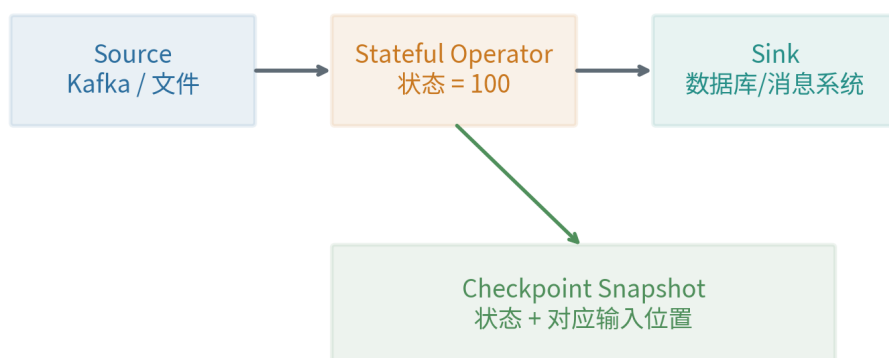
业务	需要保存的状态
累计计数	每个 key 当前 count
去重	已经见过哪些订单ID/事件ID
风控	账号最近若干次交易、失败次数、地点
窗口聚合	窗口当前 sum/count/max 等中间值
模式检测	已经匹配到规则的第几步

keyBy 为什么重要？

因为很多状态是“按 Key 隔离”的：用户 A 的状态不能和用户 B 混在一起。keyBy 会让同一个 Key 的事件落到同一个逻辑 Keyed State 分区，从而让状态操作具有清晰归属。

十、Checkpoint：流任务运行半年，机器坏了怎么办？

Checkpoint：状态很多，机器坏了怎么办？



故障后：恢复最近一致快照 + 从对应输入位置继续回放，而不是“所有数据从头算一遍”。

如果一个流任务运行了 100 天，状态里已经积累了大量信息。第 101 天某台机器损坏，如果只能“从第一天重算”，显然不可接受。

Checkpoint 的核心思想是定期生成一致状态快照。这个快照不仅要知道“状态是多少”，还要和输入流的位置对应起来。发生故障后，系统恢复最近一次成功快照，并从相应输入位置继续。

可以先把它记成：

Checkpoint $\approx$ “分布式游戏存档”

存档内容不只是角色血量 (State)
还要知道剧情进行到哪里 (Input Position)

官方文档强调，Checkpoint 让 Flink

能恢复状态及对应流位置，从而让恢复后的执行具有与无故障执行一致的语义基础。

十一、Exactly-once 到底是什么意思？先不要神化

“Exactly-once” 经常被误解成“现实世界里任何东西都绝对只发生一次”。更准确地说，它讨论的是：在故障与恢复后，计算状态和外部结果能否表现得像每条事件只被计算一次。

语义	简单理解
At-most-once	最多一次：可能丢，但一般不会重复处理
At-least-once	至少一次：尽量不丢，但失败重放可能导致重复
Exactly-once	从业务/状态语义看，故障恢复后结果等价于恰处理好一次

但要真正实现端到端 Exactly-once，不只是 Flink 自己有 Checkpoint 就够了：Source 需要支持一致恢复/回放，Sink 也要支持事务、两阶段提交或幂等等机制。

因此以后看到“Exactly-once”时，第一反应应该是问：范围是什么？只说内部状态，还是 Source → Flink → Sink 的端到端语义？

十二、Backpressure: 当下游来不及处理, 会发生什么?

在无界流里, Source 不会因为你计算慢就自动停止产生现实事件。如果下游 Operator 长期处理不过来, 数据就会积压。Flink 会让压力沿数据流向上游传播, 这就是 Backpressure。

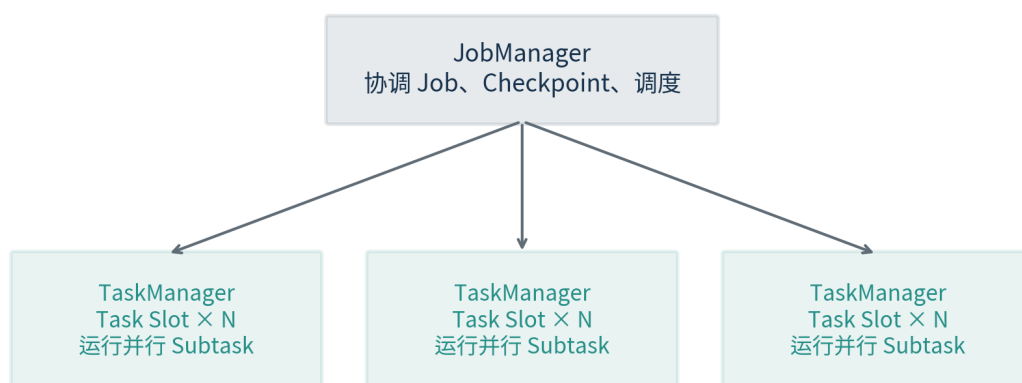
```
Source 很快
  ↓ 100万条/s
Operator A
  ↓ 100万条/s
Operator B 只能处理 20万条/s
  ↓
Sink
```

结果: B 前面开始积压 → 压力逐渐传回上游

Backpressure 不是“一个异常错误”, 而是流系统非常重要的运行信号。它可能说明某个算子太慢、外部数据库写入慢、网络不足、Key 倾斜或并行度不足。

12.1 Flink 高层架构

Flink 高层运行架构: JobManager 与 TaskManager



Operator 并行化后会产生多个 Subtask；不同 Subtask 可分布在不同 TaskManager/Slot 上运行。

第一讲只需要先认识: JobManager 负责协调与调度, TaskManager 真正运行并行 Subtask, Task Slot 表示可用于执行任务的资源槽位。后续课程会专门展开。

十三、把 MapReduce、Spark、Flink 放在同一张表里

维度	MapReduce	Spark	Flink
主要设计中心	离线批处理	通用批处理/SQL/分析	连续流处理 + 批流统一
计算组织	Map→Shuffle→Reduce	DAG→Stage→Task	持续 Dataflow / Operator
状态	不是核心编程抽象	可做，但不是最初设计中心	一等公民，深度结合运行时
时间语义	通常不突出	Structured Streaming 支持时间语义	Event Time / Watermark 是核心概念
容错直觉	Task 重试/中间结果重算	Lineage + 重算/Checkpoint	State Checkpoint + 流位置恢复
典型场景	大规模离线统计	ETL、SQL、交互分析、批处理	实时风控、监控、持续聚合、复杂流

这张表不是“优劣排名”。真正的工程问题是：你的数据是有界还是无界？需要多低延迟？状态有多复杂？对事件时间和故障一致性的要求多高？

第一阶段最重要的迁移能力，是把你已经学会的 Partition、Shuffle、并行 Task、状态恢复这些概念迁移到 Flink，而不是把旧知识全部扔掉。

十四、版本与学习策略：先学稳定概念，不追实验 API

截至 2026 年 9 月，Apache Flink 2.3.0 是最新稳定主版本之一。Flink 2.x 正在继续演进新的 API，但官方稳定文档中仍明确标注 DataStream API V2 为实验阶段。

因此本课程策略是：

优先掌握	暂时不追
Bounded / Unbounded	实验性 API 的所有新接口细节
Event Time / Watermark	版本之间零碎参数差异
Window / State / Timer	一次性背大量类名
Checkpoint / Savepoint / Exactly-once	为了“最新”而破坏已有实验环境
Parallelism / Slot / Backpressure	冷门 Connector 的细节

原因很简单：上面左侧这些概念跨版本非常稳定，是你未来阅读 Flink 论文、官方文档和排错日志的真正底层语言。

如果你当前虚拟机已经安装了较老的 Flink 版本，也无需为了本讲立即升级。本讲几乎全部是跨版本通用原理。

十五、不写代码的第一个思维实验：实时订单大屏

场景：电商平台希望实时显示“最近 1 分钟每个商品的支付金额”，每 5 秒更新一次；移动网络可能导致订单事件最多迟到 10 秒。

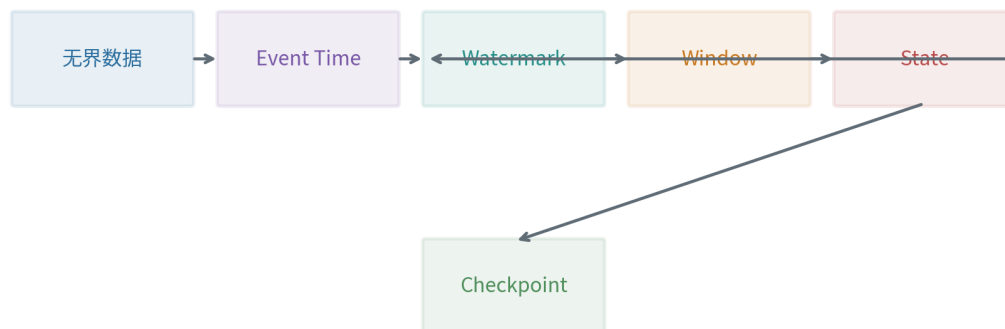
请把它拆成 Flink 概念：

问题	Flink 概念
数据会不会结束？	不会 → Unbounded Stream
按业务发生时间还是服务器到达时间？	Event Time
如何容忍最多 10 秒乱序？	Watermark Strategy
统计“最近 1 分钟、每 5 秒更新”	Sliding Window
每个商品分别统计	keyBy(product_id)
窗口内要记住什么	State (sum / count 等)
机器故障如何恢复	Checkpoint
下游数据库写得慢怎么办	观察 Backpressure / Sink 性能

你会发现：Flink 的那些“抽象概念”其实不是为了复杂而复杂，而是每一个都对应流处理中的一个现实困难。

十六、本讲最终知识地图

Flink 第一讲最终知识地图



数据不停 → 必须按“时间”理解事件 → 乱序需要 Watermark → 无限流需要 Window → 业务需要 State → 状态必须靠 Checkpoint 容错

把第一讲压缩成一句因果链：

数据不会结束 → 不能等“全部到齐” → 必须持续处理 → 业务又要求按照真实发生时间计算 → 于是出现乱序 → 需要 Watermark 推进事件时间 → 无限流需要 Window 切片 → 业务逻辑需要 State 记住过去 → State 又必须通过 Checkpoint 才能在故障后恢复。

这条链比任何 API 名称都重要。只要这条链理解了，后面学代码会容易很多。

十七、常见误区、练习与答案

17.1 常见误区

误区	纠正
Flink 就是更快的 Spark	不准确。核心差异是设计中心与流处理语义。
流处理就是每秒跑一次批任务	不准确。真正流处理关注无界数据、状态、事件时间和持续运行。
Watermark 等于“所有旧事件都绝对到了”	不准确。它表达事件时间进度，晚到事件仍可能存在。
Window 只是切时间	不完整。Window 还涉及触发、状态、迟到数据等。
Checkpoint 就是把输出文件备份一下	不准确。核心是状态与输入位置的一致恢复。
Exactly-once 是 Flink 一个人就能保证	端到端还依赖 Source/Sink 的能力与配置。

17.2 练习题

- 1) 为什么“昨天全天订单统计”天然是 bounded，而“实时支付流”天然是 unbounded？
- 2) 事件 10:00:03 发生，但 10:00:20 才到系统，按 Processing Time 与 Event Time 会分别落在哪个时间段？
- 3) Watermark 太保守与太激进分别有什么代价？
- 4) 为什么“同一账号 1 分钟内连续失败 5 次”一定需要 State？
- 5) 为什么端到端 Exactly-once 还要关心 Sink？

17.3 参考答案

- 1) 前者有确定终点，后者持续产生；2) Processing Time 按 10:00:20，Event Time 按 10:00:03；3) 保守会增加等待延迟，激进会增加 Late Event；4) 因为必须记住此前失败次数与时间；5) 即使 Flink 内部状态一致，如果 Sink 在恢复后重复写入，外部业务结果仍可能重复。

十八、这一讲之后你应该会什么？

最低闭环能力	是否应该做到
能解释 Spark 与 Flink 的设计中心差异	是
能区分 bounded / unbounded	是
能解释 Event Time 与 Processing Time	是
能用自己的话解释 Watermark	是
能说出 Window 为什么存在	是
能解释 State 为什么是流计算核心	是
能解释 Checkpoint 在恢复什么	是
知道 Exactly-once 有“范围”问题	是

下一讲：Flink 第2讲 – 从 Source 到 Sink 的 DataStream 执行模型

下一讲开始进入真正的程序结构：ExecutionEnvironment /

StreamExecutionEnvironment、Source、Transformation、keyBy、Sink、Operator

Chain、Parallelism、Subtask，以及一个 Flink Job 提交以后内部发生什么。

官方资料核对：Apache Flink Stable Docs – Learn Flink Overview; Event Time / Watermark 文档; Checkpointing 文档; Apache Flink 2.3.0 Release Announcement。